

Machine Learning Day03 hw1 보고서

작성자: Doyeon Park

작성일: 2026.09.25

목차

1. 머신러닝이란?
2. 인공신경망의 전체 구조
 3. 퍼셉트론
 4. 활성화 함수
 5. 순전파
 6. 손실함수
 7. 경사하강법
 8. 역전파
9. 신경망 학습 전체 과정
10. PyTorch

1. 머신러닝이란?

머신러닝(Machine Learning)은 사람이 문제를 해결하기 위한 모든 규칙을 프로그램으로 작성하는 대신, 주어진 데이터를 이용하여 컴퓨터가 필요한 규칙이나 패턴을 학습하도록 하는 방법이다. 기존의 규칙 기반 프로그램에서는 사람이 데이터를 처리하는 규칙을 코드로 직접 작성하지만, 머신러닝에서는 데이터를 이용하여 모델이 규칙을 찾아내고 새로운 입력에 대한 결과를 예측한다.

머신러닝은 학습 방법에 따라 크게 지도학습(Supervised Learning), 비지도학습(Unsupervised Learning), 강화학습(Reinforcement Learning)으로 구분할 수 있다.

지도학습은 입력 데이터와 함께 정답에 해당하는 라벨(Label)을 제공하여 학습시키는 방법이다. 예를 들어 고양이와 강아지 사진을 분류하는 문제에서는 이미지와 함께 해당 이미지가 고양이인지 강아지인지에 대한 정답을 제공한다. 모델은 많은 입력과 정답의 관계를 학습하여 새로운 이미지가 들어왔을 때 어떤 종류인지 예측한다.

비지도학습은 정답 없이 데이터만 제공하는 방식이다. 모델은 데이터 내부에 존재하는 유사성이나 구조를 찾아 데이터를 여러 그룹으로 나누거나 중요한 특징을 찾아낸다.

강화학습은 정답을 직접 제공하는 대신, 모델이 수행한 행동의 결과에 따라 보상(Reward)을 제공한다. 모델은 시행착오를 반복하며 더 큰 보상을 얻을 수 있는 행동을 학습한다. 예시로 휴머노이드의 보행 학습이 있다.

이 중에서도 이 보고서에서는 신경망의 구조와 퍼셉트론, 활성화 함수, 손실 함수, 경사 하강법, 순전파와 역전파가 어떻게 하나의 학습 과정으로 연결되는지를 정리할 것이다. 또한 각각의 개념이 실제 PyTorch 에서 어떤 함수와 클래스로 구현되는지도 살펴보겠다.

2. 인공신경망의 전체 구조

개별적인 개념을 설명하기 전에 인공신경망(Artificial Neural Network)의 전체적인 구조를 설명하겠다.

인공신경망은 여러 개의 뉴런 또는 노드를 층(Layer)형태로 연결한 구조이다. 신경망은 크게 입력층(Input Layer), 은닉층(Hidden Layer), 출력층(Output Layer)으로 구분할 수 있다. 입력층은 데이터를 받아들이고, 은닉층은 입력 데이터에서 특징을 추출하고 조합하며, 출력층은 최종적인 예측 결과를 만든다. 은닉층이 여러 개 존재하는 신경망을 심층신경망(Deep Neural Network, DNN)이라고 한다.

신경망의 전체적인 데이터 흐름은 다음과 같이 생각해볼 수 있다.

입력값 → 은닉층 → 은닉층 → 출력층 → 최종 예상값

예를 들어 두 개의 입력 x_1, x_2 와 두 개의 은닉 노드가 있다고 가정하면 두 은닉 노드는 동일한 입력을 받을 수 있다. 그러나 각각의 연결에는 서로 다른 가중치(Weight)가 존재하기 때문에 두 노드가 동일한 결과를 계산하는 것은 아니다.

첫 번째 노드가

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

을 계산한다면 두 번째 노드는

$$z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

를 계산할 수 있다.

두 노드는 같은 입력을 받더라도 w_{11} , w_{12} 와 w_{21} , w_{22} 가 서로 다르기 때문에 입력을 서로 다른 방식으로 해석할 수 있다.

이것이 신경망에서 여러 노드를 사용하는 중요한 이유 중 하나이다. 여러 노드가 서로 다른 가중치를 학습하면서 입력 데이터의 서로 다른 특징에 반응할 수 있고, 다음 층에서는 이 특징들을 다시 조합하여 더욱 복잡한 특징을 표현할 수 있다.

예를 들어 이미지 인식 문제를 단순화하여 생각하면 초기 층의 노드는 선이나 경계와 같은 비교적 단순한 특징에 반응하고, 이후 층에서는 여러 특징을 조합하여 더 복잡한 형태를 표현할 수 있다. 다만 특정 노드가 처음부터 ‘눈 담당’ 또는 ‘귀 담당’으로 지정되는 것은 아니다. 학습 과정에서 각 노드의 가중치가 달라지면서 결과적으로 서로 다른 특징에 반응하게 된다.

3. 퍼셉트론

3.1 퍼셉트론의 개념

퍼셉트론(Perceptron)은 인공신경망을 이해하기 위한 가장 기본적인 모델 중 하나이다.

입력이 세 개라면 퍼셉트론에서는 우선

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

를 계산한다.

여기서 각 기호의 의미는 다음과 같다.

x_i : 입력값
 w_i : 각 입력의 가중치
 b : 편향
 z : 가중합

이후 z 의 값을 기준으로 최종 출력을 결정한다.

3.2 가중치

가중치는 각 입력이 결과에 얼마나 영향을 주는지를 조절한다.

예를 들어

$$x_1=2, x_2=5$$

이고,

$$w_1=3, w_2=0.2$$

라고 하자.

편향을 0 이라고 하면

$$z=3(2)+0.2(5)$$

이므로

$$z=6+1=7$$

이 된다.

이 계산에서 x_1 보다 x_2 값이 크지만 최종결과에 더 큰 영향을 준 것은 가중치가 더 큰 x_1 이다.

가중치는 이처럼 모델이 특정 입력을 얼마나 중요하게 반영할지를 결정하는 파라미터라고 볼 수 있다.

3.3 편향

가중치만 존재하면 판단 기준을 자유롭게 이동시키는 데 제한이 생긴다.

가장 단순한 경우를 생각하여

$$z=wx$$

라고 하면 w 가 양수일 때 $z=0$ 이 되는 위치는 항상 $x=0$ 이다.

반면 편향을 추가하여

$$z=wx+b$$

로 만들면 $z=0$ 의 기준점을 이동시킬 수 있다.

예를 들어

$$w=1, b=-10$$

이면

$$z=x-10$$

이 된다.

Step 함수를 이용하여 $z \geq 0$ 일 때 1을 출력한다면

$$x-10 \geq 0$$

즉,

$$x \geq 10$$

부터 출력이 1 이 된다.

편향은 단순히 가중치와 별도로 더해지는 숫자가 아니라 뉴런의 판단 기준을 이동시키는 역할을 한다.

3.4 퍼셉트론의 학습

퍼셉트론의 중요한 특징은 처음부터 완벽한 가중치를 가지고 있는 것이 아니라 예측 결과와 정답을 비교하면서 가중치를 수정한다는 것이다.

퍼셉트론의 가중치 수정 규칙을 다음과 같다.

$$w \leftarrow w + \eta(y - \hat{y})x$$

여기서

$$\begin{aligned} y &= \text{실제 정답} \\ \hat{y} &= \text{퍼셉트론의 예측} \\ \eta &= \text{학습률} \end{aligned}$$

이다.

만약 정답과 예측이 같다면

$$y - \hat{y} = 0$$

이므로 가중치는 변하지 않는다.

반대로 실제 정답이 1 인데 퍼셉트론이 0 을 출력했다면

$$y - \hat{y} = 1$$

이 되어 가중치를 출력이 증가하는 방향으로 수정한다.

실제 정답은 0 인데 퍼셉트론이 1 을 출력했다면

$$y - \hat{y} = -1$$

이 되어 반대 방향으로 가중치를 수정한다.

3.5 퍼셉트론 학습의 예시

현재 가중치가

$$w = 0.5$$

이고, 입력이

$$x=2$$

라고 가정한다.

정답은

$$y=1$$

이지만 모델이

$$\hat{y}=0$$

을 출력했다고 하자.

학습률을

$$\eta=0.1$$

로 설정하면

$$w \leftarrow 0.5 + 0.1(1-0)(2)$$

이므로

$$w \leftarrow 0.7$$

이 된다.

즉 기존 가중치가 0.5 에서 0.7 로 수정된다.

이 과정을 반복하면서 퍼셉트론은 데이터에 더 적절한 가중치를 찾아간다.

4. 활성화 함수

4.1 활성화 함수의 위치

하나의 뉴런에서는 우선 여러 입력에 각각 가중치를 곱하고 편향을 더하여

$$z = w_1 x_1 + w_2 x_2 + \dots + b$$

를 계산한다.

하지만 일반적인 신경망에서는 이 값을 그대로 다음 층으로 보내지 않고 활성화 함수를 적용한다.

$$a = f(z)$$

즉 한 뉴런의 계산 과정을 순서대로 나타내면 **입력 → 가중치 계산 → z → 활성화 함수 → a** 가 된다.

여기서 z 는 활성화 함수를 적용하기 전의 값이고, a 는 활성화 함수를 통과한 뒤 다음 층으로 전달되는 값이다.

4.2 활성화 함수가 필요한 이유

만약 활성화 함수가 없다고 가정해 보자.

첫번째 층에서

$$z_1 = 2x + 1$$

을 계산하고, 두 번째 층에서

$$z_2 = 3z_1 + 2$$

를 계산한다고 한다.

첫 번째 식을 두 번째 식에 대입하면

$$z_2 = 3(2x + 1) + 2$$

이고 이를 정리하면

$$z_2 = 6x + 5$$

가 된다.

두 개의 층을 사용했지만 결과적으로 하나의 선형식과 같은 형태가 된다.

층을 더 많이 쌓더라도 중간 계산이 모두 선형 변환이라면 여러 선형 변환을 다시 하나의 선형 변환으로 표현할 수 있다.

따라서 신경망에서는 중간에 비선형 활성화 함수를 넣는다.

비선형 함수가 들어가면 단순히 여러 선형식을 하나로 합칠 수 없게 되며, 또한 여러 뉴런과 층을 조합하여 훨씬 복잡한 입력과 출력의 관계를 표현할 수 있게 된다.

4.3 Step 함수

Step 함수는 퍼셉트론에서 사용했던 가장 단순한 활성화 함수이다.

$$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

즉 입력 z 가 0 이상인지 아닌지만 확인하여 0 또는 1을 출력한다.

예를 들어

$$z = -3 \rightarrow 0$$

$$z = 0.5 \rightarrow 1$$

이 된다.

이 함수는 결과를 명확하게 두 가지로 나눌 수 있다는 장점이 있지만, 대부분의 구간에서 미분값이 0 이기 때문에 뒤에서 설명할 경사하강법과 역전파를 이용한 신경망 학습에는 적합하지 않다

4.4 Sigmoid 함수

Sigmoid 함수는 다음과 같이 정의된다

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid 함수의 출력은 항상 0 과 1 사이에 존재한다

입력값이 매우 큰 양수이면 출력이 1 에 가까워지고, 매우 큰 음수이면 0 에 가까워진다. $z=0$ 에서는

$$\sigma(0) = 0.5$$

가 된다.

이러한 특성 때문에 이진 분류 모델의 출력 등에서 사용될 수 있다.

그러나 입력의 절댓값이 매우 커지면 함수의 기울기가 0 에 가까워진다. 신경망이 깊을 경우 이러한 작은 기울기가 역전파 과정에서 계속 곱해지면서 앞쪽 층으로 전달되는 기울기가 매우 작아질 수 있는데, 이를 기울기 소실 문제라고 한다.

4.5 Tanh 함수

Tanh 함수는 Sigmoid 와 비슷한 곡선 형태를 가지지만 출력 범위가

$$-1 \leq f(x) \leq 1$$

이다.

Sigmoid 가 0 과 1 사이의 값을 출력하는 것과 달리 Tanh 는 출력의 중심이 0 이라는 특징이 있다.

Tanh 가 Sigmoid 보다 이러한 점에서 유리하지만, 입력값이 큰 양수 또는 음수가 되면 여전히 기울기가 작아지는 문제가 있다.

4.6 ReLU 함수

ReLU 는 현대 신경망에서 매우 자주 사용되는 활성화 함수 중 하나이다.

$$f(z) = \max(0, z)$$

즉, $z < 0$ 이면 0 을 출력하고, $z \geq 0$ 이면 입력값을 그대로 출력한다.

예를 들어

$$f(-3) = 0$$

$$f(2) = 2$$

가 된다.

ReLU 는 계산 방법이 매우 단순하고 양수 영역에서는 기울기가 1 이기 때문에 학습에 유리하지만, 음수 영역에서는 출력과 기울기가 모두 0 이기 때문에 특정 뉴런이 계속 음수 값만 받게 되면 학습이 거의 이루어지지 않을 수 있다. 이를 흔히 Dying ReLU 문제라고 한다

4.7 Leaky ReLU

Leaky ReLU 는 ReLU 의 음수 영역이 완전히 0 이 되는 문제를 완화하기 위해 나온 함수로 다음과 같다.

$$f(z) = \max(0.1z, z)$$

즉, 양수에서는 일반적인 ReLU 와 마찬가지로 값을 그대로 출력하고, 음수에서는 완전히 0 으로 만들지 않고 작은 비율을 남긴다.

예를 들어 0.1 을 기준으로 하면

$$z = -5$$

일 때

$$f(z) = -0.5$$

가 된다.

따라서 음수 영역에서도 작은 기울기가 남아 있어 ReLU 에서 발생할 수 있는 뉴런의 학습 정지 문제를 완화할 수 있다.

5. 순전파

5.1 순전파의 개념

순전파는 입력 데이터가 신경망의 입력층에서 시작하여 은닉층과 출력층을 차례로 지나면서 최종 예측값을 계산하는 과정이다.

앞에서 설명한 것처럼 하나의 뉴런은 이전 층에서 받은 입력값에 각각 가중치를 곱하고 편향을 더하여 가중합 z 를 계산한다.

$$z = Wx + b$$

그 후 계산된 z 에 활성화 함수를 적용하여 출력값 a 를 만든다.

$$a = f(z)$$

이렇게 만들어진 a 는 현재 뉴런의 계산 결과이면서 동시에 다음 층 뉴런의 입력값이 된다.

따라서 여러 층으로 구성된 신경망에서는

입력 $\rightarrow$ 가중합 $\rightarrow$ 활성화함수 $\rightarrow$ 다음층 $\rightarrow \dots$ 최종 출력

의 계산이 반복된다.

$$\begin{aligned} z^{(l)} &= W^{(l)} a^{(l-1)} + b^{(l)} \\ a^{(l)} &= f(z^{(l)}) \end{aligned}$$

여기서 위첨자 l 은 몇 번째 층인지를 나타낸다.

즉, $a^{(l-1)}$ 은 이전 층에서 넘어온 값이며, 현재 층의 가중치 $W^{(l)}$ 과 편향 $b^{(l)}$ 을 이용하여 $z^{(l)}$ 을 계산한다. 이후 활성화 함수를 적용하여 현재 층의 출력 $a^{(l)}$ 을 만들고, 이 값이 다시 다음 층의 입력으로 사용된다.

5.2 순전파에서의 가중치와 활성화 함수의 역할

순전파 과정에서는 현재 신경망이 가지고 있는 가중치와 편향을 이용하여 결과를 계산한다.

예를 들어 이전 층에서 a_1, a_2 라는 두 개의 값이 다음 뉴런에 전달되었다고 하자.

해당 뉴런의 가중치가 w_1, w_2 라면 먼저

$$z = w_1 a_1 + w_2 a_2 + b$$

를 계산한다.

이후 ReLU 를 활성화 함수로 사용한다면

$$a = \max(0, z)$$

를 계산한다.

따라서 순전파에서는 가중치와 편향으로 입력을 조합하고, 활성화 함수로 그 결과를 변환하는 과정이 각 층마다 반복된다.

중요한 점은 순전파 과정에서 가중치를 새롭게 구하거나 수정하는 것이 아니라는 것이다. 순전파에서는 현재 가지고 있는 가중치와 편향을 그대로 이용하여 모델의 예측값을 계산한다.

가중치의 수정은 이후 정답과 예측값을 비교하여 손실을 계산하고, 역전파와 경사하강법을 수행하는 과정에서 이루어진다.

5.3 순전파와 예측값

신경망의 마지막 층에서 나온 결과를 일반적으로 모델의 예측값이라고 하며 $\hat{y}$ 로 표현할 수 있다.

여기서

$$y$$

는 실제 정답이고,

$$\hat{y}$$

는 신경망이 계산한 예측값이다.

순전파가 끝났다고 해서 학습이 완료된 것은 아니다. 현재 가중치가 적절하지 않다면 예측값과 실제 정답이 크게 다를 수 있다.

예를 들어 실제 정답이

$$y=7$$

인데 순전파 결과

$$\hat{y}=5$$

가 나왔다면 모델의 예측에는 오차가 존재하므로 다음단계에서는

$$y$$

와

$$\hat{y}$$

를 비교하여 얼마나 틀렸는지를 숫자로 나타낼 필요가 있다.

이때 사용되는 것이 손실함수라는 것이다.

6. 손실함수

6.1 손실 함수의 개념

앞의 순전파 과정에서는 현재 신경망이 가지고 있는 가중치와 편향을 이용하여 최종 예측값 $\hat{y}$ 를 계산하였다. 그러나 예측값이 나왔다는 것만으로는 모델이 얼마나 정확한지 알 수 없다.

따라서 모델의 예측값 $\hat{y}$ 와 실제 정답 y 를 비교하여 예측이 얼마나 틀렸는지를 하나의 숫자로 표현할 필요가 있다.

이때 사용하는 것이 손실함수이다.

예를 들어 정답이

$$y=10$$

이고 모델의 예측값이

$$\hat{y}=9$$

라면 오차는 비교적 작다고 볼 수 있는 반면,

$$\hat{y}=2$$

라고 예측했다면 정답과 훨씬 큰 차이가 존재한다.

손실 함수는 이러한 차이를 숫자로 표현하여 모델이 현재 얼마나 잘못 예측하고 있는지 판단할 수 있도록 한다.

6.2 회귀 문제의 손실함수

6.3.1 MSE

MSE 는 평균 제곱 오차이며 다음과 같이 표현할 수 있다.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

정답과 예측값의 차이를 구한 뒤 이를 제곱하고, 여러 데이터에 대해 평균을 계산한다.

MSE 의 중요한 특징은 오차를 제곱한다는 점이다.

따라서 오차가 커질수록 손실값이 훨씬 크게 증가한다.

또한 함수가 매끄러워 미분하기 편리하다는 장점이 있지만, 매우 큰 오차가 존재하는 경우 그 값의 영향을 크게 받기 때문에 이상치에 민감하다는 특징이 있다.

6.3.2 MAE

MAE 는 평균 절대 오차이며 다음과 같이 표현된다.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MSE 와 달리 오차를 제곱하지 않고 절댓값을 이용한다.

MAE 에서는 오차의 크기가 증가하더라도 손실이 오차에 비례하여 증가하기 때문에 MSE 처럼 큰 오차의 영향을 지나치게 크게 받지 않는다.

다만 오차가 0 인 지점에서 미분이 매끄럽게 정의되지 않는다는 특징이 있다.

6.4 분류 문제의 손실함수

회귀 문제에서는 정답과 예측값의 숫자 차이를 비교하면 되지만, 분류 문제는 조금 다르다.

예를 들어 고양이와 강아지를 분류한다고 하면 최종 결과는 단순히 어떤 숫자에 가까운지를 보는 것이 아니라 정답 클래스에 얼마나 높은 확률을 부여했는지를 판단해야 한다.

Binary Cross Entropy 는 두 개의 클래스를 구분하는 이진 분류 문제에서 사용할 수 있고, Categorical Cross Entropy 는 여러 개의 클래스 중 하나를 선택하는 다중 분류 문제에서 사용할 수 있다.

따라서 분류 문제의 손실 함수는 단순히 숫자 차이만 보는 것이 아니라 정답 클래스에 모델이 얼마나 높은 확신을 주었는지를 평가한다고 볼 수 있다.

객체 검출 모델인 YOLO 와 같이 더 복잡한 문제에서는 박스 위치, 클래스, 객체성 등 여러 종류의 손실을 함께 사용할 수도 있다.

7. 경사하강법

7.1 경사하강법의 개념

앞에서 손실 함수를 이용하여 모델의 예측값 $\hat{y}$ 와 실제 정답 y 가 얼마나 다른지를 손실값 L 로 표현하였다.

그러나 손실값을 계산하는 것만으로는 학습이 이루어지지 않는다. 모델의 예측을 개선하기 위해서는 손실값이 작아지도록 신경망 내부의 가중치 w 와 편향 b 를 수정해야 한다.

이때 사용되는 대표적인 방법이 경사하강법이다.

가중치에 대한 기본적인 갱신식은 다음과 같다.

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

여기서

w : 현재 가중치

L : 손실

$\frac{\partial L}{\partial w}$: 가중치에 대한 손실의 기울기

η : 학습률

이다.

경사하강법의 목적은 한 번의 계산으로 완벽한 가중치를 구하는 것이 아니라, 현재 위치에서 손실이 감소하는 방향을 찾아 가중치를 조금씩 수정하는 것이다

7.2 기울기의 의미

경사하강법을 이해하기 위해서는

$$\frac{\partial L}{\partial w}$$

을 이해해야 한다.

이 값은 현재 가중치 w 를 조금 변화시켰을 때 손실 L 이 어느 방향으로 변화하는지를 나타낸다

예를 들어

$$\frac{\partial L}{\partial w} > 0$$

이라면 w 를 증가시키는 방향으로 움직일수록 손실이 증가하는 방향이라는 의미이다.

따라서 손실을 감소시키기 위해서는 반대 방향인 w 를 감소시키는 방향으로 이동해야 한다.

반대로

$$\frac{\partial L}{\partial w} < 0$$

이라면 w 를 증가시키는 방향으로 이동할수록 손실이 감소하는 방향이라는 의미이므로 w 를 증가시켜야 한다.

따라서 다음 식

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

에서 마이너스 기호가 사용되는 이유는 기울기가 증가하는 방향의 반대로 이동하여 손실을 감소시키기 위해서이다.

8. 역전파

8.1 역전파의 개념

앞에서 경사하강법을 이용하여 가중치를 수정하려면 각 가중치에 대한 손실의 기울기

$$\frac{\partial L}{\partial w}$$

가 필요하다는 것을 알 수 있었다.

신경망에 가중치가 하나만 존재한다면 기울기를 계산하는 것이 비교적 간단할 수 있다. 그러나 실제 심층신경망에서는 수많은 뉴런이 여러 층으로 연결되어 있으며, 그에 따라 학습해야 하는 가중치와 편향의 개수도 매우 많아진다.

이 문제를 해결하기 위해 사용하는 방법이 바로 역전파이다.

역전파는 최종 출력에서 계산된 손실을 기준으로 출력층에서 입력층 방향으로 거꾸로 이동하면서 각 가중치가 손실에 미치는 영향을 계산하는 방법이다.

역전파의 목적은 가중치를 직접 수정하는 것이 아니라,

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \dots$$

와 같이 각 가중치에 대한 손실의 기울기를 계산하는 것이다

8.2 역전파의 계산 순서

순전파에서는 입력값이 다음과 같은 방향으로 이동한다.

$$\text{입력} \rightarrow \text{은닉층} \rightarrow \text{은닉층} \rightarrow \text{출력} \rightarrow \hat{y}$$

마지막으로 예측값 $\hat{y}$ 와 실제 정답 y 를 비교하여 손실 L 을 계산한다.

즉 손실은 신경망의 가장 마지막 부분에서 계산된다.

그런데 앞쪽 층에 존재하는 가중치는 손실에 직접 연결되어 있지 않다.

예를 들어 다음과 같은 매우 단순한 신경망을 생각할 수 있다.

$$x \rightarrow w_1 \rightarrow a \rightarrow w_2 \rightarrow \hat{y} \rightarrow L$$

여기서 w_2 는 최종 출력 $\hat{y}$ 에 가까이 있기 때문에 w_2 의 변화가 손실에 미치는 영향을 비교적 직접적으로 계산할 수 있다.

반면 w_1 을 변화시키면 먼저 중간값 a 가 변한다.

그리고 a 가 변하면 다음 층의 계산 결과가 달라지고, 그 결과 최종 예측값 $\hat{y}$ 이 변한다. 마지막으로 $\hat{y}$ 이 변하면서 손실 L 이 변한다.

즉,

$$w_1 \rightarrow \text{중간값} \rightarrow \hat{y} \rightarrow L$$

처럼 여러 단계를 거쳐 손실에 영향을 준다.

따라서 앞쪽 가중치가 손실에 어떤 영향을 주었는지 계산하려면 손실이 있는 마지막 부분에서 시작하여

$$L \rightarrow \text{출력층} \rightarrow \text{은닉층} \rightarrow \text{앞쪽 가중치}$$

의 순서로 영향을 추적하는 것이 효율적이다.

이때문에 역방향으로 전파한다는 이름을 사용한다.

8.3 연쇄법칙

하나의 가중치 w 에 대한 손실의 기울기를 다음과 같이 표현할 수 있다.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \times \frac{\partial a}{\partial z} \times \frac{\partial z}{\partial w}$$

이 식을 앞에서 배운 뉴런의 구조와 연결하여 생각해보자

뉴런에서는 먼저

$$z = wx + b$$

을 계산하고,

활성화 함수를 적용하여

$$a=f(z)$$

를 만든다.

그리고 이 값들이 다음 층을 지나 최종적으로 손실 L 에 영향을 준다.

따라서 관계는

$$w \rightarrow z \rightarrow a \rightarrow L$$

의 형태로 이어진다.

각 부분의 의미를 살펴보면,

$$\frac{\partial z}{\partial w}$$

는 가중치 w 를 조금 변화시켰을 때 z 가 얼마나 변하는 지를 나타낸다.

$$\frac{\partial a}{\partial z}$$

는 z 가 변화했을 때 활성화 함수의 출력 a 가 얼마나 변하는지를 나타낸다 .

마지막으로

$$\frac{\partial L}{\partial a}$$

는 a 가 변했을 때 최종 손실 L 이 얼마나 변하는지를 나타낸다.

이 세 값을 곱하면 결국

$$\frac{\partial L}{\partial w}$$

즉 처음의 가중치 w 가 최종 손실에 얼마나 영향을 주는지를 계산할 수 있게 된다.

9. 신경망 학습 과정 정리

전체적인 흐름을 정리하면 다음과 같다.

입력 데이터

→ 순전파

→ 예측값 $\hat{y}$

→ 정답 y 와 비교

→ 손실 L

→ 역전파

→ $\frac{\partial L}{\partial w}$ 계산

→ 경사하강법으로 w, b 수정

→ 순전파

이 과정을 반복하면서 모델의 예측값이 실제 정답에 가까워지도록 가중치와 편향이 조정된다.

10. PyTorch에서의 구현

10.1 nn.Linear와 가중치 및 편향

신경망의 한 층에서는 일반적으로 다음과 같은 계산을 수행한다.

$$z = Wx + b$$

PyTorch에서는 이 계산을 주로

`nn.Linear()`

클래스를 이용하여 구현한다.

예를 들어

```
import torch.nn as nn
```

```
layer = nn.Linear(2, 3)
```

이라고 하면 2 개의 입력 특징을 받아 3 개의 출력값을 생성하는 선형층이 만들어진다.

이때 내부에는 학습 가능한 가중치와 편향이 존재한다.

즉 앞에서 설명했던 W , b 에 해당하는 값들이 `nn.Linear` 내부의 파라미터로 저장된다.

10.2 활성화함수

`nn.Linear` 를 이용하여

$$z = Wx + b$$

를 계산한 뒤에는 활성화 함수를 적용할 수 있다.

다음은 PyTorch 에서 사용할 수 있는 대표적인 활성화 함수이다.

```
nn.ReLU()
nn.Sigmoid()
nn.Tanh()
nn.LeakyReLU()
```

예를 들어 ReLU 를 사용하는 신경망은 다음과 같이 구성할 수 있다.

```
model = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)
```

이 코드에서

`nn.Linear(2,4)`는

$$z = Wx + b$$

계산을 수행하고

`nn.ReLU()`는

$$a = f(z)$$

를 수행한다.

10.3 손실함수

PyTorch 는 torch.nn 에서 다양한 손실함수를 제공한다.

MSE 는 다음과 같이 사용할 수 있다.

```
loss_fn = nn.MSELoss()
```

MAE 는 다음 클래스를 이용할 수 있다.

```
loss_fn = nn.L1Loss()
```

다중 분류 문제에서는 대표적으로

```
loss_fn = nn.CrossEntropyLoss()
```

를 사용할 수 있다.

이진 분류에서는

```
nn.BCELoss()  
nn.BCEWithLogitsLoss()
```

등을 사용할 수 있다.

10.4 loss.backward()와 역전파

역전파는 각 가중치에 대한 손실의 기울기 $\frac{\partial L}{\partial w}$ 를 계산하는 과정으로

PyTorch 에서는

```
loss.backward()
```

를 호출하여 사용할 수 있다.

PyTorch 의 torch.autograd 는 연산 과정을 추적하고, 연쇄법칙을 이용하여 각 학습 가능한 파라미터에 대한 기울기를 자동으로 계산한다.

즉,

$$\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_3}, \dots$$

을 Pytorch 가 자동으로 계산해준다.

10.5 Optimizer 와 가중치 갱신

역전파로 기울기를 계산한 뒤에는 실제 가중치를 수정해야 한다.

PyTorch에서는 이를 위해 torch.optim 패키지의 Optimizer 를 사용한다.

예를 들어 SGD 를 사용하려면 다음과 같이 작성할 수 있다

```
optimizer = torch.optim.SGD(  
    model.parameters(),  
    lr = 0.01  
)
```

여기서

```
model.parameters()
```

는 모델 내부의 학습 가능한 가중치와 편향을 optimizer 에 전달하며 lr=0.01 은 학습률을 의미한다.

참고로 PyTorch 의 SGD 클래스는 Stochastic Gradient Descent 를 구현하며, 필요하면 Momentum 도 추가할 수 있다

또 다른 대표적인 optimizer 로 Adam 이 있다

```
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=0.001  
)
```

Adam 은 기울기의 이동 평균과 제곱값 등을 이용하여 각 파라미터의 갱신 크기를 조절하는 optimizer 이다.

10.6 optimizer.step()

역전파가 끝난 뒤 실제 가중치를 수정하려면 optimizer.step 을 실행한다.

optimizer.step() 은 loss.backward() 에서 계산된 각 파라미터의 기울기를 이용하여 실제 가중치와 편향을 갱신한다.

즉, 앞에서 배운 경사 하강법의

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}$$

과 같은 역할을 수행하는 함수이다.

10.7 optimizer.zero_grad()

PyTorch에서는 기울기가 기본적으로 새로운 값으로 완전히 대체되는 것이 아니라 기존 기울기에 누적된다.

만약 기울기를 초기화하지 않으면 이전 학습에서 계산된 기울기와 현재 계산된 기울기가 계속 더해질 수 있기 때문에 의도하지 않은 학습 결과가 발생할 수 있다.

따라서 다음 학습을 시작하기 전에 기존 기울기를 초기화할 필요가 있다.

이를 위하여

```
optimizer.zero_grad()
```

를 사용한다.